App Development with Swift — Associate

Complete Study Guide: Learn It, Drill It, Pass It

Built from the official Certiport objective domains (2025) and Certiport's published exam tutorial. Read Part 0–1 first (5 min), then study Parts 2–7 domain by domain, then do Part 8 (practice) and Part 9 (traps) the night before.

PART 0 — THE EXAM (know the game)

Item	**What is published**
Provider	Certiport (a Pearson VUE business), Apple-branded
Duration	50 minutes
Questions	**40–45** (Certiport's own Associate tutorial screen shows 40; other training providers list 45. Expect roughly 40–45 and budget for 45.)
Passing score	**700 / 1000** (about 70%). Scoring is scaled, so don't try to compute exact question counts. Aim for 85%+ in practice.
Language	English
Results	Instant, shown on screen after you finish
Question types (confirmed by Certiport tutorial)	**Single-answer** (radio buttons), **Multiple-answer** (checkboxes, "Select Two/Three"), **Hot spot** (click a spot on an image/screenshot)
Tools in the exam	**Strikeout** (Ctrl+click on Mac / right-click on PC crosses out an option), **Flag for Review**, **Review screen** (shows skipped + flagged), **Calculator**, hide-able timer

Item	What is published
Before the exam starts	3 demographic questions (not scored, timer separate)

What the Associate adds over "Certified User": the Associate covers the social side (design cycle, data privacy, legal/ethical/socioeconomic impact, accessibility) *plus* Swift and SwiftUI. Don't skip Domain 1 — it is easy marks.

How the questions come (the pattern)

Objective verbs tell you the style. Look at the verbs in the official PDF: *summarize, assess, differentiate, recognize, select, evaluate, calculate, predict, interpret, explain, identify, create*. That translates to:

1. **"What is the output / value?"** — short Swift snippet, you trace it by hand. (Domain 3: operators, loops, arrays, functions, structs.) This is the biggest scoring bucket.
2. **"Which code produces this UI?" / "What does this code display?"** — SwiftUI snippet with stacks and modifiers; sometimes a hot spot on a screenshot. (Domain 4.)
3. **"Which line has the error / what type of error?"** — syntax vs run-time, read the message. (Domain 5.)
4. **"Which is the correct ...?"** — definitions and best practice: design cycle order, file types, accessibility, privacy. (Domains 1–2.)
5. **Select-THREE/TWO** questions — you must get *all* correct picks. Read the count in the question.

(Points 1–3 are my reading of the objective verbs plus the confirmed question formats; Certiport does not publish domain weights or the actual question pool, so treat this as strong guidance, not a guarantee.)

Time plan

- 50 min ÷ 45 questions ≈ **65 seconds each**.
- Pass 1 (35 min): answer everything you're sure of. **Flag** anything that needs more than ~90 sec.
- Pass 2 (12 min): go through flagged ones with the review screen.
- Last 3 min: make sure **no question is unanswered** (there's no penalty stated for guessing — never leave blanks).
- Use **strikeout** on every question: cross out 2 wrong options, then choose. Wrong options are usually wrong for one obvious reason (wrong keyword, missing `$`, wrong order).

- Use the **calculator** for operator questions only after you've decided how Swift's integer rules work (see Domain 3) — the calculator doesn't know integer division!

PART 1 — DOMAIN 1: PLANNING AND DESIGN

1.1 The design cycle

Brainstorm → Plan → Prototype → Evaluate (then iterate: feedback goes back to brainstorm/plan).

Stage	What you do	Example
Brainstorm	Generate ideas, identify the problem and the users, list features, no idea is rejected yet	"A study-timer app for students"
Plan	Pick the ideas, define features, users, screens, data; sketch/wireframe, storyboard, flowchart	Screen list, user flow diagram
Prototype	Build a quick, testable version (paper sketch, clickable mock-up, or first SwiftUI build)	Basic SwiftUI screens
Evaluate	Test with real users, collect feedback, measure against goals, then improve	Usability test, bug reports

Exam tip: "Which stage involves getting user feedback?" → **Evaluate**. "Which stage creates a working early version?" → **Prototype**. "Sketching screens and features" → **Plan**. "Generating many ideas" → **Brainstorm**.

1.2 Protecting and compromising sensitive data

1.2.1 Sharing personal and application information

- **Personal information (PII):** name, address, phone, email, birth date, location, photos, contacts, health data, financial data, passwords, ID numbers.
- **Application information:** usage data, device IDs, analytics, purchase history, in-app behavior.
- Users share data through **permissions** (camera, microphone, location, contacts, photos). iOS asks the user; the developer must explain why using a usage description in **Info.plist**.
- Good practice: **data minimization** (collect only what you need), ask permission only when needed, be transparent (privacy policy), give users control (delete/opt out).

1.2.2 Security challenges (how data gets compromised)

- **Phishing / social engineering** — tricking users into revealing info.
- **Weak or reused passwords**; no multi-factor authentication.
- **Unencrypted storage or network traffic** (use HTTPS, encryption).
- **Data breaches** — attackers steal databases.
- **Malware, insecure third-party libraries**, unpatched software.
- **Oversharing / public Wi-Fi / lost devices.**

Protections: strong passwords + **2FA**, **encryption**, HTTPS, secure login, regular updates, least-privilege permissions, don't hard-code secrets in source code.

1.2.3 Legal, ethical and socioeconomic impacts

- **Legal:** privacy laws (e.g., GDPR in EU, COPPA for children in US, India's DPDP Act 2023), copyright/licensing of images/code, terms of service, App Store guidelines and privacy labels.
- **Ethical:** informed **consent**, honesty about data use, avoiding **bias** in algorithms, not manipulating users (addictive design), protecting children.
- **Socioeconomic:** the **digital divide** (unequal access to devices/internet), cost of apps/devices, apps creating or removing jobs, access to services (banking, education, health), cultural impact of apps and social media.

1.3 Accessibility in visual design

Assess a design with these checks:

- **Color contrast** — text must stand out from the background (light gray on white fails).
- **Don't rely on color alone** — add icons, text or patterns (e.g., red/green only is a problem for color-blind users).
- **Text size / Dynamic Type** — text should scale with the user's settings; avoid tiny fixed sizes.

- **VoiceOver** — images and controls need meaningful labels (`.accessibilityLabel("Play button")`); decorative images can be hidden from VoiceOver.
- **Touch target size** — buttons at least about **44×44 points**.
- **Motion** — avoid excessive animation (Reduce Motion setting); **captions** for audio/video.
- Consistent layout, clear labels, simple navigation.

```
Image(systemName: "play.fill")

    .accessibilityLabel("Play")
```

PART 2 — DOMAIN 2: XCODE PROJECT NAVIGATION

2.1 Basic file types

File / folder	What it is
`*.swift`	Swift source code (e.g., `ContentView.swift`, `MyApp.swift`)
`MyApp.swift`	App entry point: the struct marked `@main` that conforms to `App`, contains a `WindowGroup`
`ContentView.swift`	The default first SwiftUI view
`Assets.xcassets`	**Asset catalog**: images, colors, app icon, symbols
`Info.plist`	App configuration and permission descriptions (in newer Xcode, edited in target's Info tab)
`*.xcodeproj`	The Xcode project file (settings, targets, file references)
`*.xcworkspace`	Groups multiple projects/packages
`*.playground` / Swift Playgrounds `.swiftpm`	Experimentation space / Playgrounds app package

File / folder	What it is
`*.storyboard`, `*.xib`	Legacy UIKit interface files (not used in SwiftUI)
`Preview Content`	Assets used only for previews
Media files `.png` `.jpg` `.pdf` `.svg` `.heic`	Images

2.2 Recognizing assets after import

- Imported images/colors appear in **Assets.xcassets** as **image sets** or **color sets** with their names.
- Image sets can hold **1x, 2x, 3x** versions (different screen densities).
- The **AppIcon** set holds the app icon.
- **SF Symbols** are Apple's built-in icons (not in your asset catalog): `Image(systemName: "star.fill")`.
- Use an asset by its **exact name string** (case-sensitive): `Image("beach")`, `Color("Brand")`.

2.3 Importing and using an asset

1. **Drag and drop** the file into `Assets.xcassets` in the Project navigator (or click **+** at the bottom of the asset catalog → Image Set, then drop images in).
2. Rename the image set (name = what you type in code).
3. Use it:

```
Image("beach")

    .resizable()

    .scaledToFit()

    .frame(width: 200)
```

Other route: **File ▸ Add Files to "Project"…** for non-asset files (e.g., data files, fonts); tick **Copy items if needed** and the right **target**.

2.4 Configuring areas of the UI (Xcode layout)

Area	Location	Purpose	Shortcut
Navigator area	Left	Project navigator (files), Source control, Find, Issue navigator (errors/warnings), Debug navigator	⌘1 (project), ⌘0 toggle left
Editor area	Center	Where you write code	–
Canvas (Preview)	Right of editor	Live SwiftUI preview	⌥⌘↩ toggle
Inspector area	Right	File inspector, Attributes inspector (modifiers on selected view)	⌥⌘0 toggle right
Library	Toolbar **+** button	Views, modifiers, code snippets, media	⇧⌘L
Debug area / Console	Bottom	`print` output, variable values, errors at runtime	⇧⌘Y
Toolbar	Top	**Run ▸ (⌘R)**, **Stop ■ (⌘.)**, scheme/device picker (choose simulator), status	
Simulator	Separate app	Runs your app on virtual iPhone/iPad	

Things you can do: run on a simulator, switch device in the canvas, use **Live Preview** to interact, click a view in the canvas → use **Attributes inspector** to add modifiers, use **Issue navigator** to see all errors, open **Library** to drag a view in.

Preview code:

#Preview {

```
    ContentView()
}
```

(older style: `struct ContentView_Previews: PreviewProvider { static var previews: some View { ContentView() } }`)

PART 3 — DOMAIN 3: SWIFT LANGUAGE USAGE (the biggest domain — master this)

3.1 Functions

```
func greet(_ name: String, from city: String, times count: Int = 1) -> String {
    return "Hi \(name) from \(city)"
}

greet("Sam", from: "Surat")              // count uses default value 1

greet("Sam", from: "Surat", times: 3)
```

Anatomy: `func` keyword, function **name**, **parameters** (`externalLabel internalName: Type`), **return type** after `->`, **body**.

- **Argument label** = what you write in the call. **Parameter name** = what you use inside the function.
- `_` in front means **no label at the call site**.
- If only one name is given (`func add(a: Int, b: Int)`), it is both label and parameter name → call `add(a: 1, b: 2)`.
- No `->` means returns nothing (`Void`).
- **Single-expression function** can omit `return`: `func double(_ n: Int) -> Int { n * 2 }`.
- Default parameter values can be skipped in the call.
- Return values may be ignored or stored: `let result = double(4)`.
- Return can be multiple values as a **tuple** (less commonly tested): `-> (min: Int, max: Int)`.

Trace method: substitute arguments, compute the body, apply return.

```
func area(width: Int, height: Int) -> Int { width * height }

area(width: 3, height: 4) + area(width: 2, height: 5)   // 12 + 10 = 22
```

3.2 Operators

Type	Operators	Notes
Arithmetic	`+ - * / %`	`/` on two Ints → **integer division, truncates toward zero** (`7 / 2 = 3`, `-7 / 2 = -3`). `%` remainder (`7 % 3 = 1`).
Double math	`7.0 / 2 = 3.5`	If either side is Double, result is Double (only when both types are already Double — see below)
Compound	`+= -= *= /= %=`	`x += 3` means `x = x + 3`
Comparison	`== != < > <= >=`	Result is **Bool**
Logical	`&&` (and), `||` (or), `!` (not)	`!` binds tightest, then `&&`, then `||`
Ternary	`cond ? a : b`	`let s = 7 > 5 ? "big" : "small"`
Range	`1...5` (closed, 1–5), `1..<5` (half-open, 1–4)	
Nil-coalescing	`a ?? b`	uses `b` if `a` is nil
String	`+` joins strings	`"A" + "B" = "AB"`

Precedence: `* / %` before `+ -`. Use parentheses to be sure. Example: `2 + 3 * 4 = 14`, `(2 + 3) * 4 = 20`.

Type mixing is NOT allowed: `let a = 5; let b = 2.5; a * b` → **error**. Fix: `Double(a) * b` = 12.5. (Swift never converts Int to Double for you.)

String interpolation: `"Total: \(price * 2)"`.

Trace practice:

- `17 % 5 + 7 / 2` → `2 + 3` = **5**
- `var x = 5; x += 3; x *= 2` → 8 → **16**
- `true && !false || false` → `true && true || false` → **true**
- `10 / 4` = 2, `10 % 4` = 2, `10.0 / 4` = 2.5

3.3 Structures (structs)

```
struct Player {

    // 3.3.1 Declare properties

    var name: String

    var score: Int = 0          // default value

    let id: Int                 // constant property

    var doubleScore: Int { score * 2 }   // computed property

    // 3.3.3 Define methods

    func describe() -> String { "\(name): \(score)" }

    mutating func addPoints(_ pts: Int) { score += pts }   // must be 'mutating' to change properties

}

// 3.3.2 Initialize: memberwise initializer is automatic (in property order)

var p1 = Player(name: "Ava", score: 5, id: 1)   // 3.3.4 Create an instance

let p2 = Player(name: "Ben", id: 2)             // score uses default 0

// 3.3.5 Use an instance

p1.addPoints(10)          // p1.score = 15

print(p1.describe())      // "Ava: 15"

p2.addPoints(1)           // ERROR: p2 is a let constant, method is mutating
```

Key facts:

- **Properties** store data; **methods** are functions inside a type.
- **Memberwise initializer** is generated automatically; arguments are labeled with property names, in declaration order. Properties with defaults may be omitted.
- **Custom initializer** — if you write your own `init`, use `self.name = name`. Then the automatic memberwise init is no longer provided (unless defined in an extension).
- `let` property can't be changed after init. `let` **instance** of a struct → **none** of its `var` properties can be changed.
- **Structs are value types:** assigning copies.

```
var a = Player(name: "A", id: 1)

var b = a

b.score = 99

a.score      // still 0 — b is an independent copy
```

- Access with dot syntax: `p1.name`, `p1.describe()`.
- Naming: types **UpperCamelCase** (`Player`), properties/methods **lowerCamelCase**.
- `self` refers to the current instance.

3.4 Arrays

```
var fruits = ["Apple", "Banana", "Cherry"]     // 3.4.1 declare + initialize, inferred [String]

var scores: [Int] = [10, 20, 30]               // explicit type

var empty = [String]()                         // empty array (also: var empty: [String] = [])
```

3.4.2 Index (starts at 0):

```
fruits[0]              // "Apple"

fruits[2] = "Cherry Pie"   // modify

fruits[fruits.count - 1]   // last element

fruits[3]              // RUN-TIME CRASH: Index out of range
```

3.4.3 Properties and methods:

Code	Result
`fruits.count`	number of elements
`fruits.isEmpty`	Bool
`fruits.first` / `.last`	optional first/last element
`fruits.append("Fig")`	add at end
`fruits.insert("Kiwi", at: 1)`	insert at index, shifts others
`fruits.remove(at: 0)`	remove & return element at index
`fruits.removeLast()`	remove last
`fruits.removeAll()`	empty it
`fruits.contains("Apple")`	Bool
`scores.sorted()`	returns new sorted array (original unchanged)
`scores.sort()`	sorts in place
`scores.reversed()`	reversed sequence
`scores.min()` / `.max()`	optional
`fruits + ["Fig"]`	new combined array
`fruits += ["Fig"]`	append contents

- Array declared with **`let`** is immutable: `append`, `remove`, changing an element → **error**.
- All elements must be the **same type**.
- Tracing example:

```
var a = [4, 8, 15, 16]

a.remove(at: 1)      // [4, 15, 16]

a.append(23)         // [4, 15, 16, 23]

a.insert(7, at: 0)   // [7, 4, 15, 16, 23]

a[2]                 // 15,  a.count = 5
```

3.5 Control flow

3.5.1 Loops

```
for i in 1...3 { print(i) }              // 1 2 3

for i in 1..<3 { print(i) }              // 1 2

for fruit in fruits { print(fruit) }     // each element

for (index, fruit) in fruits.enumerated() { }

for i in stride(from: 0, to: 10, by: 3) { }   // 0 3 6 9  (to = excludes 10)

for i in stride(from: 0, through: 10, by: 5) { } // 0 5 10 (through = includes)

var n = 1

while n < 20 { n *= 3 }                  // n: 1 → 3 → 9 → 27, stops; n = 27

var c = 10

repeat { c += 1 } while c < 5            // runs ONCE first, c = 11
```

- `break` exits the loop immediately. `continue` skips to the next iteration.
- `for _ in 1...5` when you don't need the counter.
- Tracing tips: write a small table of variable values per iteration.

3.5.2 Conditionals

```
if score >= 90 { "A" } else if score >= 80 { "B" } else { "C" }

switch score {

case 90...100: print("A")

case 80..<90:  print("B")

default:       print("C")       // switch MUST be exhaustive

}
```

- `switch` doesn't fall through; no `break` needed.
- `else if` chain stops at the **first true** branch.
- `if` condition must be a **Bool** (no `if 1`).
- Combine with `&&`, `||`.

3.6 Constants, variables and data types

`let`	**constant** — value can't change (compile-time error if you try)
`var`	**variable** — value can change
Prefer `let` unless you need to change it	

Core types: `Int` (whole numbers), `Double` (decimals, default for decimal literals), `Float`, `String`, `Character`, `Bool`, arrays.

3.6.2 Type inference: Swift figures out the type from the value.

```
let a = 3        // Int

let b = 3.0      // Double

let c = "3"      // String

let d = true     // Bool

let e = [1, 2]   // [Int]
```

3.6.3 Explicit typing: `let price: Double = 5` → 5.0; `var name: String`. Use explicit typing when there's no initial value, or you want a different type (e.g., Double from whole number).

Conversions: `Double(intValue)`, `Int(doubleValue)` (truncates: `Int(3.9)` = 3), `String(number)`.

3.7 Naming

- **lowerCamelCase** for variables, constants, functions, methods, properties: `userName`, `totalScore`, `calculateArea`.
- **UpperCamelCase** for types (struct, class, enum) and views: `Player`, `ContentView`.

- Valid identifier rules: can contain letters, digits, `_`, and unicode; **cannot start with a digit**, **no spaces**, **no hyphens/symbols** like `-`, `@`, `!`; **cannot be a reserved keyword** (`class`, `func`, `let`, `var`, `if`, `for`...) without backticks; **case-sensitive** (`score` ≠ `Score`).
- Valid: `userName`, `_temp`, `player2`, `π`. Invalid: `2ndPlace`, `my-var`, `first name`, `class`.

PART 4 — DOMAIN 4: VIEW BUILDING WITH SWIFTUI

4.1 Imperative vs declarative

- **Imperative:** you write step-by-step instructions on *how* to change the UI (create a label, set its text, add it to the screen, update it when data changes) — UIKit style.
- **Declarative (SwiftUI):** you describe *what* the UI should look like for the current state; SwiftUI updates the screen automatically when state changes.

Basic SwiftUI file:

```
import SwiftUI

struct ContentView: View {                // a view is a struct conforming to View

    var body: some View {                 // body returns some View (one view)

        Text("Hello")

    }

}
```

4.2 Content views

```
Text("Hello, world!")

Image("beach")                        // from Assets

Image(systemName: "star.fill")        // SF Symbol
```

Circle() Rectangle() RoundedRectangle(cornerRadius: 12) Capsule() Ellipse()   // Shapes

Color.blue   Color(red: 0.2, green: 0.4, blue: 0.9)   // Color is also a view

- Shapes are filled with `.fill(Color.red)`; stroke with `.stroke(Color.red, lineWidth: 3)`.
- `Color.red` alone fills the available space.

4.3 Modifiers

Modifiers return a **new view** and are applied **in order, top to bottom**.

Modifier	**Purpose**
`.padding()` / `.padding(20)` / `.padding(.horizontal)`	space around the view
`.background(Color.yellow)`	color/view behind
`.frame(width: 100, height: 50)` / `.frame(maxWidth: .infinity)`	size
`.foregroundColor(.red)` (`.foregroundStyle(.red)` newer)	text/symbol/shape color
`.font(.title)` / `.largeTitle` / `.headline` / `.body` / `.system(size: 24)`	font
`.bold()` `.italic()`	style
`.resizable()`	lets an **Image** be resized (must come **first** on an Image)
`.scaledToFit()` / `.scaledToFill()` / `.aspectRatio(contentMode: .fit)`	keep aspect ratio
`.clipShape(Circle())` / `.cornerRadius(10)`	shape clipping
`.opacity(0.5)`	transparency

ORDER MATTERS (frequent exam question):

Text("Hi").padding().background(Color.blue)   // blue area INCLUDES the padding (bigger box)

Text("Hi").background(Color.blue).padding() // blue only behind the text; padding is outside

Text("Hi").frame(width: 200).background(Color.red) // red box is 200 wide

Text("Hi").background(Color.red).frame(width: 200) // red hugs the text; frame is invisible/transparent

Image rule: `Image("beach").resizable().frame(width: 100, height: 100)` — without `.resizable()`, `.frame` will not scale the picture.

4.4 Container views (stacks)

VStack(alignment: .leading, spacing: 10) { Text("A"); Text("B") } // vertical, A above B

HStack { Text("A"); Text("B") } // horizontal, A left of B

ZStack { Color.blue; Text("Top") } // layered: LATER = on top

Spacer() // flexible empty space that pushes siblings apart

- `HStack { Text("L"); Spacer(); Text("R") }` → L at far left, R at far right.
- Stacks can be **nested** (VStack containing HStacks) to build layouts.
- `alignment` for VStack: `.leading`, `.center`, `.trailing`. HStack: `.top`, `.center`, `.bottom`.
- `.padding()` on a stack pads the whole group.
- A `body` normally returns one root view — a stack when you need several.

4.5 View hierarchy

Read the code as a tree: **parent** → **children**.

```
VStack {
    Text("Title")
    HStack {
        Image(systemName: "star")
        Text("5")
    }
```

```
    Button("Go") { }

}
```

Tree: **VStack** → [**Text("Title")**, **HStack** → [**Image**, **Text("5")**], **Button**].

- Direct children of the VStack = **3** (Text, HStack, Button).
- Total views = **6** counting the VStack (or 5 counting only inside).
- Screen order: Title on top; below it a star then "5" side by side; button at the bottom.
- Modifiers wrap views but don't add extra children you count as content.

4.6 Interactive views

```
Button("Tap me") { count += 1 }                    // title + action closure

Button(action: { count += 1 }) { Text("Tap me") }    // custom label

TextField("Enter name", text: $name)                // placeholder, binding to a String

Slider(value: $level, in: 0...10, step: 1)          // level must be Double

Toggle("Notifications", isOn: $isOn)                // isOn must be Bool
```

- Controls that change a value take a **binding**: `$propertyName`.
- Slider value type is **Double**. Show it with `Text("\(Int(level))")` or `Text(level, format: .number)`.
- Toggle is on/off (Bool). TextField is text (String).

4.7 @State

```
struct CounterView: View {

    @State private var count = 0

    var body: some View {

        VStack {

            Text("Count: \(count)")

            Button("Add") { count += 1 }     // tap → state changes → body recomputed → text updates
```

```
    }

  }

}
```

- `@State` = value **owned by the view**; when it changes, SwiftUI **redraws** the view automatically.
- Must be `var`, usually `private`.
- Use `$count` to pass a **binding** to controls; use `count` (no `$`) to read the value.
- Without `@State`, changing a plain `var` inside a struct view's body causes an error ("Cannot assign to property: 'self' is immutable") or won't update the UI.
- State drives appearance:

```
@State private var isOn = false

Circle().fill(isOn ? Color.green : Color.red)

Toggle("Power", isOn: $isOn)

if isOn { Text("Enabled") }
```

- `@State` does **not** persist after the app is closed.

PART 5 — DOMAIN 5: DEBUGGING

5.1 Kinds of errors

Type	When	Example	Result
Syntax / compile-time error	while **building** (before app runs)	missing `}`, misspelled keyword/variable, wrong type, changing a `let`, missing argument label	**Red** error in Xcode; app will not build/run

Type	When	Example	Result
Run-time error	while the app is **running**	array index out of range, force-unwrap `nil` (`!`), Int division by zero, invalid conversion	App **crashes**, message in Debug console
Logic error	app runs, wrong result	using `+` instead of `-`, wrong loop bounds	No error message; you find it by testing
Warning (yellow)	build succeeds	variable never mutated, unused variable	App still runs

5.2 Interpreting common messages

Message (simplified)	Meaning / fix
`Cannot assign to value: 'x' is a 'let' constant`	Change `let` → `var`
`Cannot find 'x' in scope` (older: *use of unresolved identifier*)	Typo, or variable not declared/not in scope
`Cannot convert value of type 'String' to specified type 'Int'`	Type mismatch
`Binary operator '+' cannot be applied to operands of type 'Int' and 'Double'`	Convert one: `Double(a) + b`
`Missing argument label 'name:' in call`	Add the label
`Extra argument` / `Missing argument for parameter`	Wrong number of arguments
`Missing return in a function expected to return 'Int'`	Add `return`
`Expected '}' in ...` / `Expected ')'`	Unbalanced brackets

Message (simplified)	Meaning / fix
`Cannot use mutating member on immutable value`	Instance is `let` or method needs `var`
`Value of optional type 'String?' must be unwrapped`	Unwrap with `if let`, `??`, or `!`
`Fatal error: Index out of range`	RUN-TIME: index ≥ count or negative
`Unexpectedly found nil while unwrapping an Optional value`	RUN-TIME: `!` on nil
`Cannot assign to property: 'self' is immutable`	In a struct view you need `@State`
`Cannot find 'x' in scope` in SwiftUI `$x`	Property missing or no `@State`

How to read: Xcode shows **red circle = error, yellow triangle = warning**, in the **Issue navigator** (⌘5) and inline in the editor. Read the *first* error first — one mistake often creates several. **Fix-its** (Xcode suggestion) appear in the message.

PART 6 — RAPID-FIRE REVISION SHEET (read this the night before and morning of)

1. Design cycle: **Brainstorm** → **Plan** → **Prototype** → **Evaluate**.
2. Accessibility: contrast, Dynamic Type, VoiceOver labels, 44 pt targets, don't rely on color alone.
3. Protect data: HTTPS/encryption, strong passwords + 2FA, minimize collection, permissions with reasons.
4. Assets → **Assets.xcassets**; use by name; SF Symbols use `systemName:`.
5. Run ⌘**R**, Stop ⌘**.**, Library ⇧⌘**L**, Debug area ⇧⌘**Y**.
6. `let` = can't change. `var` = can change.
7. Int / Int **truncates**. `%` is remainder. No mixing Int and Double.
8. `1...5` includes 5. `1..<5` stops at 4. `stride(from:to:by:)` excludes `to`.
9. Arrays start at **0**. Out-of-range = **crash** (run-time).
10. `remove(at:)` shifts later elements left. `insert(_:at:)` shifts right.
11. `while` may run 0 times. `repeat-while` runs ≥ 1 time.
12. `switch` must be exhaustive, doesn't fall through.

13. Functions: `_` = no label. Call must use the labels exactly.
14. Struct: memberwise init automatic. Changing properties in a method needs **mutating**. `let` instance = fully frozen. Structs copy.
15. Names: variables `lowerCamelCase`, types `UpperCamelCase`. Can't start with digit; no spaces or hyphens; no keywords.
16. SwiftUI = **declarative**. `body` is `some View`.
17. **Modifier order matters** (`padding` then `background` ≠ `background` then `padding`).
18. `Image` needs `.resizable()` before resizing with `.frame`.
19. VStack = up/down. HStack = left/right. ZStack = front/back (last on top). Spacer pushes.
20. `TextField(text: $s)` needs String state. `Toggle(isOn: $b)` needs Bool. `Slider(value: $d)` needs Double.
21. `@State` → private var owned by the view → change triggers redraw. `$` = binding.
22. Syntax error = won't build. Run-time error = crashes while running. Logic error = wrong answer, no message.
23. Red = error, yellow = warning (warning doesn't stop the build).

PART 7 — WORKED CODE-TRACING EXAMPLES (do these by hand)

A.

```
var total = 0

for i in 1...4 { total += i * 2 }

print(total)
```

i=1→2, i=2→4 (6), i=3→6 (12), i=4→8 (20). **Output: 20**

B.

```
var names = ["Ann", "Bob", "Cat"]

names.insert("Dan", at: 1)

names.remove(at: 2)

print(names)
```

After insert: [Ann, Dan, Bob, Cat]; remove index 2 (Bob): **[Ann, Dan, Cat]**

C.

```
func calc(_ a: Int, _ b: Int = 2) -> Int { a * b + 1 }

print(calc(3) + calc(3, 4))
```

calc(3) = 3*2+1 = 7; calc(3,4) = 13. **Output: 20**

D.

```
struct Counter {
    var value = 0
    mutating func step() { value += 2 }
}
var c = Counter()
c.step(); c.step()
var d = c
d.step()
print(c.value, d.value)
```

c.value = 4; d copied (4) then step → 6. **Output: 4 6**

E.

```
let score = 72
if score >= 90 { print("A") }
else if score >= 70 { print("B") }
else if score >= 50 { print("C") }
else { print("F") }
```

First true = `>= 70`. **Output: B**

F.

```
var x = 10
while x > 0 { x -= 3 }
print(x)
```

10→7→4→1→−2 (stops). **Output: −2**

G.

```
struct ContentView: View {
    var body: some View {
        ZStack {
            Color.yellow
            VStack {
                Text("A")
                Text("B")
            }
        }
    }
}
```

Yellow fills the background; A above B centered on top of it.

H.

```
@State private var on = false
```

Toggle("Light", isOn: $on)

Text(on ? "ON" : "OFF")

Starts showing **OFF**; switching the toggle sets `on = true` → view redraws → **ON**.

PART 8 — 50 PRACTICE QUESTIONS (cover the answers, then check)

Domain 1

1. Which design-cycle stage produces an early working/test version?
2. Which TWO actions best protect user data? (A) Use HTTPS/encryption (B) Collect every possible data point "just in case" (C) Collect only the data you need (D) Store passwords in plain text
3. Your app uses the camera. What must you provide?
4. Which improves accessibility for a screen-reader user? (A) Bigger shadows (B) An accessibility label on an image button (C) Red text on green (D) Removing all text
5. The "digital divide" is a ___ impact. (A) legal (B) ethical (C) socioeconomic (D) technical

Domain 2

6. Where do you put images/colors for your app?
7. Which shortcut runs the app?
8. Which Xcode area shows the live SwiftUI preview?
9. Which file does an app's `@main` struct usually live in?
10. How do you use an SF Symbol vs. an imported image?

Domain 3

11. `func double(_ n: Int) -> Int { n * 2 }` — value of `double(4) + double(3)`?
12. Given `func add(a: Int, b: Int) -> Int`, which call is correct? (A) `add(1, 2)` (B) `add(a: 1, b: 2)` (C) `add(1, b: 2)`
13. `let r = 10 / 4 + 10 % 4` → r = ?
14. `var x = 5; x += 3; x *= 2` → x = ?
15. `let a = true, b = false; a && !b || b` → ?
16. `let s = 7 > 5 ? "big" : "small"` → s = ?
17. Given `struct P { var name: String; var age: Int }`, how do you create an instance?

18. Which keyword must precede a struct method that changes a property?
19. `var a = P(name:"A", age:1); var b = a; b.age = 9; print(a.age)` → ?
20. `let nums = [3,1,2]; nums.append(4)` → what happens?
21. `var v = [3,1,2]; let s = v.sorted(); print(v)` → ?
22. `let names = ["A","B","C"]; names[names.count]` → ?
23. `var s = 0; for i in stride(from: 0, to: 10, by: 3) { s += i }` → s = ?
24. `var n = 1; while n < 20 { n *= 3 }` → n = ?
25. `var c = 10; repeat { c += 1 } while c < 5` → c = ?
26. `let g = 85; switch g { case 90...100: print("A") case 80..<90: print("B") default: print("C") }` → ?
27. Type of `let x = 3.0`? Type of `let y = 3`?
28. `let a = 5; let b = 2.5; print(a * b)` → ?
29. Which is correct naming for a struct? (A) userProfile (B) UserProfile (C) user_profile
30. Which are INVALID identifiers? (A) `my-var` (B) `player2` (C) `2ndPlace` (D) `_temp` (E) `class` *(select three)*
31. `let n = 4; print("n is \(n * 2)")` → ?
32. `var t = 0; for i in 1..<5 { if i % 2 == 0 { t += i } }` → t = ?

Domain 4

33. SwiftUI is ___ programming.
34. In a `ZStack`, which child is displayed on top?
35. `HStack { Text("L"); Spacer(); Text("R") }` — what does it look like?
36. Difference between `.frame(width: 100).background(.red)` and `.background(.red).frame(width: 100)`?
37. What must come before `.frame` on an `Image` to resize it?
38. What does `TextField("Name", text: $name)` require for `name`?
39. What type must `level` be in `Slider(value: $level, in: 0...10)`?
40. What type is `isOn` in `Toggle("X", isOn: $isOn)`?
41. What does `@State` do?
42. In `VStack { Text("Title"); HStack { Image(systemName:"star"); Text("5") }; Button("Go"){} }` how many direct children does the VStack have?
43. What does `body` return in a SwiftUI view?
44. Which code gives text a blue background that includes the padding?
45. `@State private var count = 0` / `Button("Add"){ count += 1 }` / `Text("\(count)")` → after 3 taps the text shows?

Domain 5

46. Missing closing brace is what type of error?
47. Force-unwrapping a `nil` value is what type of error?

48. Your program builds, runs, no error, but the total is wrong. Type?
49. `Cannot assign to value: 'count' is a 'let' constant` — fix?
50. `Binary operator '+' cannot be applied to operands of type 'Int' and 'Double'` — fix?

ANSWER KEY WITH EXPLANATIONS

1. **Prototype.**
2. **A and C.** Encryption/HTTPS and data minimization.
3. **Permission request with a usage description (Info.plist) explaining why.**
4. **B.** VoiceOver reads accessibility labels.
5. **C.** Unequal access to technology is a socioeconomic issue.
6. **Assets.xcassets** (asset catalog).
7. ⌘**R** (Stop = ⌘.).
8. **Canvas** (Preview). Toggle with ⌥⌘↩.
9. **The app file, e.g., `MyApp.swift`** (struct marked `@main`, conforms to `App`).
10. **SF Symbol:** `Image(systemName: "star")`. **Imported image:** `Image("name")`.
11. **14.** 8 + 6.
12. **B.** No underscore, so labels are required: `add(a: 1, b: 2)`.
13. **4.** 10/4 = 2 (integer), 10%4 = 2 → 2 + 2.
14. **16.** 5 → 8 → 16.
15. **true.** `!false`=true → `true && true` = true → `true || false` = true.
16. **"big".**
17. `let p = P(name: "A", age: 1)` — automatic memberwise initializer.
18. `mutating`.
19. **1.** Structs are value types; `b` is a copy.
20. **Compile-time error.** `nums` is a `let` constant array.
21. **[3, 1, 2].** `sorted()` returns a new array; use `sort()` to change `v`.
22. **Run-time crash** (index out of range). Last valid index is `count - 1`.
23. **18.** 0 + 3 + 6 + 9 (`to:` excludes 10).
24. **27.** 1 → 3 → 9 → 27, then 27 < 20 is false.
25. **11.** `repeat` runs once before the check.
26. **B.** 85 is in 80..<90.
27. **Double** and **Int.**
28. **Error.** Int × Double isn't allowed. Fix: `Double(a) * b` = 12.5.
29. **B.** Types use UpperCamelCase.
30. **A, C, E.** Hyphen not allowed, can't start with a digit, `class` is a keyword.
31. **n is 8.**
32. **6.** 1..<5 = 1,2,3,4; adds 2 and 4.
33. **Declarative.**
34. **The last one listed** (layers go back to front).

35. **L on the far left, R on the far right**, empty space between.
36. **First:** red box exactly 100 wide (background applied to the framed view). **Second:** red hugs the text only; frame doesn't show color. *Modifier order matters.*
37. **`.resizable()`.**
38. **A `String` `@State` (or other binding) variable**, passed with `$`.
39. **Double.**
40. **Bool.**
41. **Lets a view own a value; when the value changes SwiftUI redraws the view.**
42. **3** (Text, HStack, Button).
43. **`some View`** — a single view (usually a stack when you need several).
44. **`Text("Hi").padding().background(.blue)`.**
45. **3.**
46. **Syntax (compile-time) error.**
47. **Run-time error** (app crashes).
48. **Logic error.**
49. **Change `let count` to `var count`** (and in a view, make it `@State`).
50. **Convert the types to match**, e.g., `Double(intValue) + doubleValue`.

Scoring yourself: 45+/50 = ready. 38–44 = review the domains you missed, redo. Below 38 = redo Parts 3–4, they carry the most marks.

PART 9 — TOP 25 TRAPS THAT COST MARKS

1. Forgetting **integer division truncates** (`7/2 = 3`, not 3.5).
2. Off-by-one: `1..<5` does **not** include 5; `stride(to:)` excludes the end, `stride(through:)` includes it.
3. Counting array positions from 1 instead of **0**.
4. Forgetting that `remove(at:)` shifts the rest, changing later indexes.
5. Thinking `sorted()` changes the original array (it doesn't; `sort()` does).
6. Missing that a `let` array/struct **can't be modified**.
7. Assuming `repeat-while` can run 0 times (it runs at least once).
8. Choosing `while` semantics wrong: check the condition **before** each pass.
9. Misreading **argument labels** and `_`.
10. Forgetting `mutating` on struct methods that change properties.
11. Thinking a struct copy shares data (it doesn't).
12. Mixing Int and Double without converting.
13. Confusing **camelCase** (`userName`) with PascalCase (`UserName`) — types use Pascal, everything else lowerCamel.
14. Identifiers starting with a number, or with `-`/spaces.
15. **Modifier order** (`padding`/`background`/`frame`).

16. Forgetting `.resizable()` before `.frame` on Image.
17. Mixing up VStack/HStack (V = vertical). Later ZStack items are on **top**.
18. Forgetting the `$` for bindings on TextField/Toggle/Slider (and adding `$` where you just read a value).
19. Slider value must be **Double**, not Int.
20. `@State` needed for changing view data; it's a **private var**.
21. Calling something a syntax error when it only fails at run time (index out of range is **run-time**).
22. Ignoring the **count** in "Select THREE" questions.
23. Reading **only the first option** — check all four; the exam loves a near-identical distractor.
24. In hot-spot questions, read the instruction (which element? which line?) before clicking.
25. Spending 3 minutes on one question — **flag and move on**.

PART 10 — YOUR STUDY SCHEDULE

If you have 3 days

- Day 1: Parts 1–3 (read + Part 7 examples A–F). Redo Part 8 Q1–32.
- Day 2: Parts 4–5 (type every SwiftUI example in Xcode/Swift Playgrounds and change the modifier order to see the difference). Redo Q33–50.
- Day 3: Part 6 sheet, Part 9 traps, full Part 8 timed (50 questions in 50 minutes), review mistakes.

If you have 1 day

- 2 hrs: Domain 3 (Part 3) with code trace practice.
- 1.5 hrs: Domain 4 (Part 4), typing examples in Xcode.
- 30 min: Domains 1, 2, 5 (Parts 1, 2, 5).
- 1 hr: Part 8 timed. Then Part 6 + Part 9 before sleep.

Exam morning: read Part 6 and Part 9 only. Arrive early, tidy desk/system, use strikeout on every question, flag long ones, never leave a blank.

Practice tip: the official Apple *App Development with Swift* course and Certiport's *GMetrix* practice tests are the closest thing to the real exam interface; if your Tinkering Hub has a voucher/practice-test license, use one timed run before the real attempt.

Good luck. Score 700 to pass; aim for 900.

Yes — for 60 seconds, we need to cut the explanation and keep the strongest hook + your current work + why leadership matters to you.

60-second version

> Hi, I'm Manav Sinh, a B.Tech CSE-AIML student at Parul University.

We often think technology is about coding, AI and innovation. But I've started asking myself — who makes the decisions that shape how technology is actually used?

Currently, I'm involved in the Swift Coding Club, working on technology and AI projects, and helping other students learn and build. These experiences have taught me that building technology is only one part of the picture — communication, teamwork and decision-making matter just as much.

That's why I want to be part of the IIMUN Delhi Leadership Tour. I want to hear different perspectives on governance, business, entrepreneurship and public policy, interact with people who make real-world decisions, and learn from their experiences.

I want to bring those perspectives back into what I build, how I work with people, and how I lead.

Thank you.

This should land around 55–65 seconds, depending on your speaking speed. For the video, speak naturally rather than trying to recite it perfectly.

For this application, I'd make the video feel like a short personal introduction, not a formal college presentation.

🎥 Recommended setup

Length: 50–60 seconds
Orientation: Vertical (9:16) is safest unless the form specifies otherwise
Camera: Phone rear camera if someone can record you; otherwise front camera
Location: Quiet, clean background — preferably somewhere connected to your work/studies
Lighting: Face a window or light source. Don't stand with the light behind you.
Clothing: Simple, clean, semi-formal — a plain shirt/T-shirt is fine.

🎬 Shot structure

0–5 sec — Hook Look directly into the camera:

> "We often think technology is about coding, AI and innovation. But I've started asking myself — who makes the decisions that shape how technology is actually used?"

This immediately gives them something to listen to.

5–15 sec — Who you are

> "Hi, I'm Manav Sinh, a B.Tech CSE-AIML student at Parul University."

Then briefly mention what you're actually involved in.

15–35 sec — Your reason

Talk about your involvement in the Swift Coding Club, technology/AI projects and helping students. Connect that experience to leadership:

> “These experiences have taught me that building technology is only one part of the picture — communication, teamwork and decision-making matter just as much.”

35–52 sec — Why this tour

Talk about wanting to understand governance, business, entrepreneurship and public policy, and hear perspectives different from your own.

52–60 sec — Strong ending

> “I want to bring those perspectives back into what I build, how I work with people, and how I lead.”

Then simply:

> “Thank you.”

Important: don't overproduce it

You don't need:

Background music

Dramatic transitions

Lots of B-roll

Animated text

A PowerPoint

Editing effects

A clean one-take video with good eye contact and confident delivery can work very well for this type of application.

One thing I'd change from your current plan

Don't try to sound too impressive. The selection team is asking why you want the tour and what you expect to gain. Your strongest angle is:

Technology student → already involved in building/teaching → realised leadership affects technology → wants exposure to people making real decisions → wants to bring those lessons back.

That's a coherent story in 60 seconds.